

```
from pyfiglet import Figlet
from colorama import Fore, init
import time

init()

f = Figlet(font='slant')
print(Fore.CYAN + f.renderText('My App'))

print(Fore.YELLOW + "Loading", end="")
for _ in range(5):
    print(".", end="", flush=True)
    time.sleep(0.5)

print(Fore.GREEN + "\n✓ Ready to go!")
```

💡 Pro Tips

- Use emojis (✓ ✕ 🚀 🔥) — modern terminals support them
- Keep spacing clean (avoid clutter)
- Combine animation + color + ASCII for best effect
- Libraries worth exploring:
 - rich (🔥 best for modern terminal UI)
 - blessed
 - curses (advanced)

If you want, I can design a **full startup screen** (like a hacker dashboard or app splash screen) tailored to your project 🔥



write a code to auto matically push files to git repo when
placed in the directory
make this code to refer a csv file Csv file has local
directory path, GitHub repo URL
provide a sample csv file

You're basically building an **auto-sync bot** for local folders → GitHub. Clean idea. Here's

a working solution using Python + a CSV config.



How It Works

- A CSV file defines:
 - Local folder path
 - GitHub repo path (already cloned locally)
- Script watches folders for changes
- Automatically runs:
 - `git add .`
 - `git commit`
 - `git push`



Sample CSV File (`repos.csv`)

csv

```
local_path,repo_path
/Users/yourname/projects/project1,/Users/yourname/repos/project1
/Users/yourname/projects/project2,/Users/yourname/repos/project2
```

👉 Important:

- `local_path` = folder you drop files into
- `repo_path` = **already cloned Git repo locally** (not just URL)



Python Script

Install dependency:

Bash

```
pip install watchdog
```



Auto Git Sync Script

Python

Run

```
import os
import time
import subprocess
import csv
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class GitAutoPushHandler(FileSystemEventHandler):
    def __init__(self, repo_path):
        self.repo_path = repo_path
        self.last_commit_time = 0

    def on_any_event(self, event):
        # Avoid too frequent commits
        if time.time() - self.last_commit_time < 5:
            return

        print(f"🔄 Change detected in {self.repo_path}")

        try:
            subprocess.run(["git", "add", "."], cwd=self.repo_path)
            subprocess.run(
                ["git", "commit", "-m", "Auto commit"],
                cwd=self.repo_path
            )
            subprocess.run(["git", "push"], cwd=self.repo_path)

            print("✅ Changes pushed successfully!")
            self.last_commit_time = time.time()

        except Exception as e:
            print("❌ Error:", e)

def load_config(csv_file):
    configs = []
    with open(csv_file, newline='') as f:
        reader = csv.DictReader(f)
        for row in reader:
            configs.append((row['local_path'], row['repo_path']))
    return configs

def main():
    observer = Observer()
    configs = load_config("repos.csv")

    for local_path, repo_path in configs:
        if not os.path.exists(local_path):
            print(f"⚠️ Skipping missing path: {local_path}")
            ..
```

```
        continue

    event_handler = GitAutoPushHandler(repo_path)
    observer.schedule(event_handler, path=local_path, recursive=True)

    print(f"👁️👁️ Watching: {local_path} → {repo_path}")

    observer.start()

    try:
        while True:
            time.sleep(10)
    except KeyboardInterrupt:
        observer.stop()

    observer.join()

if __name__ == "__main__":
    main()
```

⚠️ Important Notes (Read This)

1. Repo must already exist locally

Run once manually:

Bash

```
git clone https://github.com/username/repo.git /path/to/repo
```

2. Git authentication

Use one of:

- SSH (git@github.com:...)
- or GitHub CLI login:

Bash

```
gh auth login
```

3. Avoid infinite loops

If your `local_path == repo_path` , it may retrigger itself.

👉 Better:

- Keep separate input folder
- OR add ignore rules

4. Improve commit messages (optional)

Replace:

Python

Run

```
"Auto commit"
```

with:

Python

Run

```
f"Auto update at {time.strftime('%Y-%m-%d %H:%M:%S')}"
```

🔥 Optional Upgrades

If you want this to feel **next level**, I can help you add:

- 🧠 Smart diff-based commits
- 📁 Sync multiple branches
- 🔔 Slack/Discord notifications
- 🖥️ GUI dashboard
- 🐳 Dockerized background service

If you want, I can convert this into a **ready-to-run macOS/Linux service** or **Windows background app** so it runs silently forever.



sample csv file

Here's a clean, ready-to-use **sample CSV file** for your setup 📌

 **repos.csv**

csv

```
local_path,repo_path
/Users/yourname/watch/project1,/Users/yourname/repos/project1
/Users/yourname/watch/project2,/Users/yourname/repos/project2
/home/ubuntu/watch/app,/home/ubuntu/repos/app
C:\Users\YourName\watch\demo,C:\Users\YourName\repos\demo
```

💡 How to Use

- `local_path` → Folder you drop/change files in
- `repo_path` → Local folder where the Git repo is already cloned

⚠️ Example (Realistic)

csv

```
local_path,repo_path
/Users/shubham/Desktop/auto-sync/projectA,/Users/shubham/git/projectA
/Users/shubham/Desktop/auto-sync/projectB,/Users/shubham/git/projectB
```

🧠 Tips

- Paths must be **absolute paths** (no `./relative`)
- On Windows:
 - Use either `\\` or `/`
 - Example:

```
C:\\Users\\Shubham\\watch\\project,C:\\Users\\Shubham\\repos\\
```

- Make sure each `repo_path` :
 - already has `.git` initialized

- is connected to a remote repo

If you want, I can also generate a **CSV dynamically from your existing folders + GitHub repos** to save setup time.



adjust the code as if csv has 3 things
local_path, repo_url, repo_name

Got it — now we'll make the script smarter so it:

- Reads: local_path, repo_url, repo_name
- Automatically **clones the repo (if not present)**
- Uses a standard base directory for repos
- Then watches + auto pushes

Updated Sample CSV (repos.csv)

csv

```
local_path,repo_url,repo_name
/Users/shubham/watch/project1,https://github.com/username/project1.git,pro
/Users/shubham/watch/project2,https://github.com/username/project2.git,pro
```

Updated Python Script (Auto Clone + Watch + Push)

Python

Run

```

import os
import time
import subprocess
import csv
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

# 📁 Base directory where all repos will be cloned
BASE_REPO_DIR = os.path.expanduser("~/auto_git_repos")

class GitAutoPushHandler(FileSystemEventHandler):
    def __init__(self, repo_path):
        self.repo_path = repo_path
        self.last_commit_time = 0

    def on_any_event(self, event):
        if time.time() - self.last_commit_time < 5:
            return

        print(f"🔄 Change detected in {self.repo_path}")

        try:
            subprocess.run(["git", "add", "."], cwd=self.repo_path)
            subprocess.run(
                ["git", "commit", "-m", f"Auto update {time.strftime('%Y-%m-%d %H:%M:%S')}",
                 cwd=self.repo_path
            )
            subprocess.run(["git", "push"], cwd=self.repo_path)

            print("✅ Changes pushed!")
            self.last_commit_time = time.time()

        except Exception as e:
            print("❌ Error:", e)

def load_config(csv_file):
    configs = []
    with open(csv_file, newline='') as f:
        reader = csv.DictReader(f)
        for row in reader:
            configs.append((row['local_path'], row['repo_url'], row['repo_name']))
    return configs

def ensure_repo_cloned(repo_url, repo_name):
    repo_path = os.path.join(BASE_REPO_DIR, repo_name)

    if not os.path.exists(repo_path):
        print(f"📥 Cloning {repo_name}...")
        subprocess.run(["git", "clone", repo_url, repo_path])

```



```
        subprocess.run(["git", "clone", repo_url, repo_path])
    else:
        print(f"✓ Repo exists: {repo_name}")

    return repo_path

def sync_files(local_path, repo_path):
    # Copy files from local_path → repo_path
    for root, dirs, files in os.walk(local_path):
        rel_path = os.path.relpath(root, local_path)
        target_dir = os.path.join(repo_path, rel_path)

        os.makedirs(target_dir, exist_ok=True)

        for file in files:
            src_file = os.path.join(root, file)
            dest_file = os.path.join(target_dir, file)

            try:
                with open(src_file, "rb") as fsrc:
                    with open(dest_file, "wb") as fdst:
                        fdst.write(fsrc.read())
            except Exception as e:
                print(f"✗ File copy error: {e}")

def main():
    os.makedirs(BASE_REPO_DIR, exist_ok=True)

    observer = Observer()
    configs = load_config("repos.csv")

    for local_path, repo_url, repo_name in configs:
        if not os.path.exists(local_path):
            print(f"⚠ Missing local path: {local_path}")
            continue

        repo_path = ensure_repo_cloned(repo_url, repo_name)

        # Initial sync
        sync_files(local_path, repo_path)

        handler = GitAutoPushHandler(repo_path)
        observer.schedule(handler, path=local_path, recursive=True)

        print(f"👁 Watching: {local_path} → {repo_name}")

    observer.start()

    try:
        while True:
```

```
        time.sleep(10)
    except KeyboardInterrupt:
        observer.stop()

    observer.join()

if __name__ == "__main__":
    main()
```

What Changed (Important)

New Features

- Auto clones repo using `repo_url`
- Uses `repo_name` to organize repos
- Copies files from `local_path` → repo
- Then commits + pushes automatically

Things to Watch Out For

1. Git auth

Make sure:

Bash

```
git push works without password
```

👉 Use SSH or credential manager

2. File syncing behavior

- This is a **one-way sync** (local → repo)
- Deletes are NOT handled (can add if you want)

3. Large folders

For heavy usage, consider:

- `rsync` instead of Python copy
- debounce events better

If You Want Next-Level

I can upgrade this into:

-  **bi-directional sync**
-  only commit changed files
-  auto delete removed files
-  Docker background service
-  GUI dashboard

Just tell me 



Not pushing files at all

Alright — if it's **not pushing at all**, the issue is almost always one of these (and your current script *does* have a couple problems). Let's fix it properly.

Root Problems in Your Current Script

1. You're watching `local_path` but committing in `repo_path`

- Changes happen in `local_path`
- But Git only sees changes inside `repo_path`
- If sync didn't run → **nothing to commit**

 Your script only copies files **once at startup**, not on every change.

2. No check if commit actually has changes

Git will silently fail like:

```
nothing to commit, working tree clean
```

❌ 3. No logging of git errors

You're not seeing why `git push` fails.

✅ FIXED VERSION (Works Properly)

This version:

- Syncs files **on every change**
- Shows git output (so you see errors)
- Only commits when there are changes

🐍 Updated Script (Reliable)

Python

Run

```
import os
import time
import subprocess
import csv
import shutil
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

BASE_REPO_DIR = os.path.expanduser("~/auto_git_repos")

class GitAutoPushHandler(FileSystemEventHandler):
    def __init__(self, local_path, repo_path):
        self.local_path = local_path
        self.repo_path = repo_path
        self.last_commit_time = 0

    def sync_files(self):
        for root, dirs, files in os.walk(self.local_path):
            rel_path = os.path.relpath(root, self.local_path)
            target_dir = os.path.join(self.repo_path, rel_path)

            os.makedirs(target_dir, exist_ok=True)

            for file in files:
                src = os.path.join(root, file)
                dst = os.path.join(target_dir, file)

                if not os.path.exists(dst) or os.path.getmtime(src) != os.path.getmtime(dst):
                    shutil.copy2(src, dst)

    def has_changes(self):
        result = subprocess.run(
            ["git", "status", "--porcelain"],
            cwd=self.repo_path,
            capture_output=True,
            text=True
        )
        return result.stdout.strip() != ""

    def git_push(self):
        subprocess.run(["git", "add", "."], cwd=self.repo_path)

        commit = subprocess.run(
            ["git", "commit", "-m", f"Auto update {time.strftime('%Y-%m-%d %H:%M:%S')}"],
            cwd=self.repo_path,
            capture_output=True,
            text=True
        )
        print(commit.stdout, commit.stderr)
```

```

        push = subprocess.run(
            ["git", "push"],
            cwd=self.repo_path,
            capture_output=True,
            text=True
        )
        print(push.stdout, push.stderr)

    def on_any_event(self, event):
        if event.is_directory:
            return

        if time.time() - self.last_commit_time < 3:
            return

        print(f"🔄 Change detected: {event.src_path}")

        try:
            self.sync_files()

            if self.has_changes():
                print("📦 Changes found, pushing...")
                self.git_push()
                print("✅ Done")
            else:
                print("ℹ️ No changes to commit")

            self.last_commit_time = time.time()

        except Exception as e:
            print("❌ Error:", e)

    def load_config(csv_file):
        configs = []
        with open(csv_file, newline='') as f:
            reader = csv.DictReader(f)
            for row in reader:
                configs.append((row['local_path'], row['repo_url'], row['repo_
        return configs

    def ensure_repo(repo_url, repo_name):
        repo_path = os.path.join(BASE_REPO_DIR, repo_name)

        if not os.path.exists(repo_path):
            print(f"⬇️ Cloning {repo_name}...")
            subprocess.run(["git", "clone", repo_url, repo_path])
        else:
            print(f"✓ Repo exists: {repo_name}")

        return repo_path

```

```
    return repo_path

def main():
    os.makedirs(BASE_REPO_DIR, exist_ok=True)

    observer = Observer()
    configs = load_config("repos.csv")

    for local_path, repo_url, repo_name in configs:
        if not os.path.exists(local_path):
            print(f"⚠ Missing: {local_path}")
            continue

        repo_path = ensure_repo(repo_url, repo_name)

        handler = GitAutoPushHandler(local_path, repo_path)
        observer.schedule(handler, path=local_path, recursive=True)

        print(f"👁 Watching {local_path} → {repo_name}")

    observer.start()

    try:
        while True:
            time.sleep(5)
    except KeyboardInterrupt:
        observer.stop()

    observer.join()

if __name__ == "__main__":
    main()
```



Debug Checklist (Do This)

Run these manually inside your repo folder:

Bash

```
git status
git push
```

If push fails:

- ✗ Authentication issue → fix with:

Bash

```
git remote -v
```

Use SSH:

Bash

```
git clone git@github.com:username/repo.git
```

Quick Test

1. Start script
2. Add a file in `local_path`
3. You should see:

```
 Change detected  
 Changes found, pushing...  
 Done
```

If STILL Not Working

Tell me:

- Your OS (Mac / Windows / Linux)
- What output you see in terminal
- Whether `git push` works manually

I'll debug it with you step-by-step.



```
[rejected]      main -> main (non-fast-forward)
error: failed to push some refs to 'https://github.com/
deka1105/TestWatcher.git'
hint: Updates were rejected because the tip of your
current branch is behind
hint: its remote counterpart. Integrate the remote
changes (e.g.
```


hint: 'git pull ...') before pushing again.

hint: See the 'Note about fast-forwards' in 'git push --help' for details.

This error is **not a bug in your script** — it's a Git sync issue:

Your local repo is **behind** the remote repo, so Git blocks the push to prevent overwriting changes.

What's Happening

Remote (GitHub) has commits that your local repo doesn't have.

Git is basically saying:

"Pull first, then push."

Quick Fix (Manual – Do This Once)

Go to your repo folder and run:

Bash

```
git pull origin main --rebase
git push
```

Proper Fix (Update Your Script)

You should **always pull before pushing** to avoid this error.

Update your `git_push()` function

Replace it with this:

Python

Run

```
def git_push(self):
    # Pull latest changes first
    print("📥 Pulling latest changes...")
    pull = subprocess.run(
        ["git", "pull", "--rebase"],
        cwd=self.repo_path,
        capture_output=True,
        text=True
    )
    print(pull.stdout, pull.stderr)

    # Add changes
    subprocess.run(["git", "add", "."], cwd=self.repo_path)

    # Commit
    commit = subprocess.run(
        ["git", "commit", "-m", f"Auto update {time.strftime('%Y-%m-%d %H: %M:%S')}"],
        cwd=self.repo_path,
        capture_output=True,
        text=True
    )
    print(commit.stdout, commit.stderr)

    # Push
    print("📤 Pushing...")
    push = subprocess.run(
        ["git", "push"],
        cwd=self.repo_path,
        capture_output=True,
        text=True
    )
    print(push.stdout, push.stderr)
```

⚠️ Important Edge Cases

🔥 1. Merge conflicts

If both local + remote modify the same file:

You'll see:

```
CONFLICT (content): Merge conflict in file.txt
```

👉 Your script will fail here. Options:

- Manual resolution (recommended)
- OR force strategy (below)

⚡ 2. Auto-resolve conflicts (optional, aggressive)

If you want your local files to **always** win:

Python

Run

```
["git", "pull", "--rebase", "-X", "theirs"]
```

⚠️ This can overwrite remote changes — use carefully.

💣 3. Force push (NOT recommended unless you know why)

Python

Run

```
["git", "push", "--force"]
```

This overwrites remote history.



Verify Fix

After updating:

1. Run script
2. Add/change a file
3. You should see:

```
⬇️ Pulling latest changes...
📦 Changes found
⬆️ Pushing...
✅ Done
```



Pro Tip (Best Practice)

For automation like this:

- Use a **dedicated branch** (e.g., `auto-sync`)
- Avoid pushing directly to `main`

If you want, I can upgrade this into a **conflict-proof sync engine (like Dropbox + Git combined)** — way more reliable than raw git commands.



 IFTTT · Sponsored ...

Automate Anything with IFTTT, No Code Required
 IFTTT makes automation easy. Connect 1000+ popular social media, business, and productivity apps.

